



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления АПК
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Компьютерные науки и интеллектуальный анализ данных

КУРСОВОЙ ПРОЕКТ

на тему: «Разработка веб-приложения для проведения викторин с
геймификацией».

Выполнил:
Студент 3 курса группы Д-Э-311
Конюхов М.В.

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва, 2026

Содержание

Введение	3
Глава 1. Теоретическая часть	5
1.1. Обзор предметной области	5
1.2. Теоретические основы геймификации	6
1.3. Анализ существующих решений	8
1.4. Функциональные требования	10
1.5. Нефункциональные требования	11
1.6. Выбор и обоснование технологического стека.....	12
Глава 2. Практическая часть	14
2.1. Проектирование базы данных.....	14
2.2. Структура проекта.....	15
2.3. Описание реализованного функционала	18
2.4. Демонстрация пользовательского интерфейса	21
2.5. Что можно добавить в систему в будущем	28
Заключение.....	31
Список использованных источников	33
Приложения	35

Введение

В связи с быстрым развитием информационных технологий и широким использованием интернета веб-приложения стали главным средством автоматизации учебных и рабочих процессов. Одним из многообещающих направлений является создание систем проверки знаний, в которых используются элементы геймификации. Такой подход повышает заинтересованность пользователей благодаря игровым элементам: начислению баллов, получению уровней, выдаче наград и ведению списков лучших участников.

Актуальность данной работы вызвана растущей потребностью в интерактивных платформах для тестирования знаний в образовании и корпоративном обучении. Обычные способы проверки знаний часто не дают достаточной мотивации участникам, тогда как игровые системы показывают значительно лучшие результаты по удержанию пользователей и качеству усвоения учебного материала.

Цель курсового проекта заключается в создании веб-приложения для проведения викторин с игровыми элементами на основе современного набора технологий.

Для выполнения поставленной цели требуется решить следующие задачи:

- Изучить правила построения информационных систем и способы создания веб-приложений;
- Проанализировать существующие решения в области платформ для тестирования и геймификации;
- Обосновать выбор инструментов и технологий, используемых при разработке;
- Разработать архитектуру системы, схему базы данных и пользовательский интерфейс;
- Создать веб-приложение с системой входа пользователей, модулем викторин и игровыми механиками;
- Проверить работоспособность разработанной системы и проанализировать полученные результаты.

Объектом исследования является процесс проектирования и создания информационных систем на основе веб-технологий.

Предметом исследования выступают методы и средства построения веб-

приложения для викторин с элементами геймификации.

Глава 1. Теоретическая часть

1.1. Обзор предметной области

Веб-викторины представляют собой интерактивные информационные системы, предназначенные для проверки знаний, формирования навыков и повышения вовлечённости пользователей. Такие приложения могут функционировать в двух режимах:

- синхронном, когда несколько участников одновременно выполняют задания под управлением ведущего;
- асинхронном, где пользователь проходит задания в удобное для себя время.

Формат вопросов варьируется от простых вариантов множественного выбора до открытых вопросов и заданий на заполнение пропусков. Выбор формата влияет на способы оценивания и анализа полученных результатов.

В научной литературе геймификация рассматривается как инструмент, который при правильном использовании способен повышать внутреннюю мотивацию пользователей, обеспечивать быструю обратную связь и поддерживать эффект «потока» – то есть оптимальное соотношение между сложностью задания и уровнем навыков участника.

Области применения веб-викторин разнообразны – от образовательных и корпоративных учреждений, до публичных мероприятий и маркетинговых кампаний.

В каждом из этих контекстов требования к функциям системы и показателям эффективности различаются. Для образовательных задач важна точность оценивания и анализ учебных достижений, тогда как для развлекательных мероприятий ключевыми являются вовлечённость аудитории и устойчивый интерес к платформе.

Основные требования предметной области включают:

- поддержку различных типов вопросов;
- надёжный учёт результатов;
- механизмы формирования рейтингов и отчётов;
- возможность масштабируемого проведения сессий в реальном времени.

Также важное значение имеют вопросы безопасности и конфиденциально-

сти данных пользователей, особенно в образовательной и корпоративной среде. Для оценки эффективности систем применяются такие метрики, как вовлечённость, частота повторного использования, доля успешно пройденных заданий и качество обучения. Это позволяет соотнести выбранные игровые механики с реальными учебными или бизнес-целями.

1.2. Теоретические основы геймификации

Геймификация определяется как применение игровых элементов и приёмов в неигровых ситуациях с целью повышения мотивации и вовлечённости пользователей [5, 6, 15]. В основе её эффективности лежат психологические и педагогические теории мотивации. Теория самоопределения выделяет три главные потребности человека:

- автономия – возможность самостоятельно выбирать действия;
- компетентность – ощущение успешности и умения справляться с задачами;
- связанность – чувство принадлежности к группе и взаимодействие с другими.

Удовлетворение этих потребностей способствует росту внутренней мотивации [15]. Концепция «потока» показывает, что наилучший опыт достигается при равновесии между сложностью задания и уровнем навыков пользователя [5]. Бихевиористские модели делают упор на роль подкрепления: поощрения усиливают желаемое поведение и делают его более частым.

Игровые механики выполняют три взаимосвязанные функции: дают обратную связь, выстраивают структуру прогресса, создают социальное взаимодействие.

В контексте викторин можно воспользоваться следующими элементами:

- очки за правильные ответы;
- бонусы за быстроту реакции;
- бонусы за серии верных ответов;
- уровни пользователей;
- достижения (награды).

Такие механики помогут пользователю ощущать прогресс и рост своей компетентности, побудят возвращаться к заданиям и повысят активность во время

прохождения викторины. Например, система очков и бонусов за быстрый ответ закрепляет оперативность и быстроту мышления, тогда как уровни и достижения поддерживают долгосрочную мотивацию, показывая шаги учебного или профессионального развития.

При разработке игровых механик важно учитывать их влияние на обучение. Механики должны усиливать учебную цель, а не заменять её. Очки и таблицы лидеров могут повышать вовлечённость, но если делать слишком большой упор на соревновательность, есть риск, что пользователи будут хуже усваивать материал. Серии правильных ответов и ежедневные задания помогают выработать привычку и регулярно заниматься, что положительно сказывается на запоминании информации. Одновременно необходимо давать обратную связь об ошибках: краткое пояснение после ответа позволяет превратить поведенческую реакцию в осознанное знание.

Правила геймификации должны быть чёткими и понятными. Пользователю необходимо ясно понимать:

- за что начисляются очки;
- как рассчитываются бонусы;
- какие условия нужно выполнить для получения достижения или повышения уровня.

Баланс наград является очень важным условием. Если давать слишком много очков за простые действия, ценность достижений снижается и мотивация падает. Если награды слишком редкие или непонятные, они не будут побуждать к действию.

Социальные аспекты геймификации сильно влияют на вовлечённость. Таблицы лидеров и групповая динамика усиливают соревновательные и кооперативные мотивы. Однако нужно учитывать, что разные пользователи по-разному реагируют на конкуренцию. Для одних людей открытое соревнование служит мотиватором. Для других оно, наоборот, снижает желание участвовать.

Поэтому полезно будет предусмотреть настройки приватности, а также механики, ориентированные на личный прогресс, а не только на сравнение с другими участниками.

Оценка эффективности геймификации должна опираться на набор показателей, связанных как с вовлечённостью, так и с учебными результатами. Ключевыми показателями являются: частота и длительность сессий, количество по-

вторных обращений к платформе, доля завершённых викторин, динамика среднего балла, сохранение знаний с течением времени.

Анализ этих данных позволяет проверить, какие игровые механики дают наибольший эффект, и при необходимости скорректировать систему наград с учётом выявленных моделей поведения.

1.3. Анализ существующих решений

Изучение действующих платформ для проведения викторин позволяет выделить общие подходы к организации интерфейса, игровым механикам и сбору данных, а также обнаружить типичные недостатки, которые обосновывают создание новой системы. Далее рассмотрены наиболее популярные сервисы с точки зрения их возможностей, технической реализации, преимуществ и ограничений.

Kahoot! [10] представляет собой инструмент для синхронных игр в реальном времени. Его интерфейс отличается простотой и направлен на удержание внимания: ведущий запускает игру, участники подключаются по коду и отвечают на вопросы мгновенно. Игровые элементы включают очки за верные и быстрые ответы, а также таблицу лидеров в рамках одной сессии. Основные достоинства – лёгкость начала работы, яркое визуальное оформление и высокая вовлечённость при очных или дистанционных занятиях. Недостатки: слабая аналитика долгосрочных результатов, ограниченная поддержка сложных типов вопросов и необходимость одновременного участия всех игроков для сохранения динамики.

Quizizz [14] поддерживает как синхронный, так и асинхронный режимы. Платформа предоставляет домашние задания, автоматическое оценивание и более подробные отчёты об успеваемости по сравнению с сервисами, ориентированными только на живые игры. Игровые механики включают очки, мемы, аватары и таблицы лидеров. Quizizz даёт больше свободы в настройке порядка прохождения и распределении материалов, однако возможности изменения логики начисления наград и сложной системы достижений остаются ограниченными. Для задач строгого контроля знаний и детального анализа данных это может быть недостаточно.

Moodle (модуль Quiz) [12] ориентирован на профессиональное использова-

ние в учебных заведениях. Ключевые преимущества – глубокая интеграция с системой управления обучением (LMS), разнообразные типы вопросов, гибкие механизмы оценки и расширение функционала через плагины. Moodle предоставляет подробную статистику по вопросам и пользователям, а также удобные средства контроля. Однако интерфейс часто считают громоздким, а настройка и администрирование требуют много времени. Игровые механики в базовой версии почти отсутствуют и добавляются отдельными расширениями.

Socrative и аналогичные системы опроса в классе предназначены для быстрого сбора ответов и простоты использования преподавателем. Они удобны для оперативной проверки усвоения материала, но не рассчитаны на накопление достижений в долгосрочной перспективе или создание разветвлённой системы уровней. Mentimeter [11] больше нацелен на интерактивные презентации и опросы; его функционал для оценки знаний и игровых механик ограничен.

Google Forms и аналогичные универсальные формы представляют собой простой способ создания тестов и опросов. Их достоинства – общедоступность, простота и интеграция с сервисами Google. Однако они почти не приспособлены для игр в реальном времени, не поддерживают динамические таблицы лидеров и не содержат встроенных игровых элементов. Аналитика сводится к базовым отчётам и требует дополнительной обработки.

С технической стороны многие коммерческие продукты используют веб-сокеты или собственные сервисы реального времени для синхронных сессий, облачные хранилища для медиафайлов и внешние системы аналитики. Платность и ограниченная открытость API в ряде решений создают препятствия для интеграции с LMS и корпоративными системами. Вопросы конфиденциальности и хранения персональных данных также различаются: образовательные платформы часто предусматривают механизмы соблюдения местного законодательства, тогда как у развлекательных сервисов политика обработки данных может быть менее строгой.

Выявленные в обзоре недостатки определяют рациональные направления для разработки нового приложения. Во-первых, отсутствует гибкая и понятная система игровых механик, которая объединяла бы краткосрочные стимулы (очки, бонусы за скорость, таблицы лидеров) и долгосрочные (уровни, достижения, ежедневные серии). Во-вторых, многие решения предлагают либо мощную аналитику без удобной геймификации, либо сильную вовлечённость без глубо-

ких средств оценки и отслеживания прогресса. В-третьих, ограниченный набор типов вопросов, слабая персонализация сложности и нехватка адаптивных сценариев затрудняют применение платформ там, где важна индивидуальная траектория обучения. Наконец, проблемы интеграции – отсутствие открытых API и сложность подключения к LMS или единой системе входа – усложняют внедрение решений в образовательную инфраструктуру.

1.4. Функциональные требования

Функциональные требования описывают поведение системы при взаимодействии с пользователем. Система должна поддерживать регистрацию и аутентификацию, как через электронную почту, так и внешних провайдеров (OAuth). Необходимо реализовать ролевую модель с разграничением доступа: администратор, автор вопросов, модератор и участник. Приложение должно предоставлять средства для создания, изменения и группировки викторин, поддерживать несколько типов вопросов (выбор одного варианта, выбор нескольких, короткий ответ, сопоставление, установление порядка) и позволять добавлять к вопросам медиафайлы. Также требуется реализовать настройку параметров сессии: выбор режима (синхронный или асинхронный), установка таймеров, весовых коэффициентов вопросов и механизмов случайной выборки.

Требования геймификации включают начисление очков за правильные ответы, бонусов за быстрое реагирование, учёт серий верных ответов, введение уровней пользователей, достижений и ежедневных серий. Правила расчёта баллов и условия получения достижений должны быть прозрачными и настраиваемыми через интерфейс администратора или автора викторины. В синхронных сессиях система должна отображать таблицу лидеров с обновлением позиций в реальном времени. Для асинхронного режима необходима корректная запись попыток и формирование отчётов по результатам.

Аналитическая подсистема должна обеспечивать как хранение, так и обобщение данных о попытках пользователей, формировать отчёты по отдельным вопросам и по общей статистике викторин. Требуется возможность получать метрики вовлечённости и успеваемости: количество завершённых сессий, средний балл, динамику серий, удержание пользователей и распределение ошибок по вопросам. Должен быть предусмотрен экспорт данных в стандартных фор-

матах для последующей обработки.

1.5. Нефункциональные требования

Нефункциональные требования охватывают производительность, масштабируемость, доступность, безопасность, конфиденциальность и удобство использования.

Производительность и масштабируемость. Система должна выдерживать одновременное участие не менее 100 активных пользователей в синхронной сессии для базовой версии (MVP) и быть масштабируемой до 1000 и более пользователей при последующем развитии. Время отклика API для типовых операций при нормальной нагрузке не должно превышать нескольких сотен миллисекунд. Обновление таблицы лидеров в ходе живой сессии должно происходить с задержкой не более одной секунды в стандартных условиях.

Доступность. Система должна обеспечивать непрерывность работы с целевым уровнем доступности, сопоставимым с показателем SLA 99.5% в производственной среде.

Безопасность. Требуется защита веб-интерфейса от распространённых уязвимостей (XSS, CSRF, SQL-инъекции), надёжное хранение паролей с применением современных алгоритмов хеширования и обязательное шифрование каналов связи (TLS). Необходимо соблюдение требований к защите персональных данных, включая возможность удаления и обезличивания записей по запросу пользователя в соответствии с действующими нормативными актами. Для социальных функций следует предусмотреть настройки приватности и возможность ограничения видимости результатов по выбору пользователя.

Доступность. Система должна обладать адаптивным дизайном для мобильных устройств и настольных компьютеров, интуитивно понятным интерфейсом для каждого типа пользователя и соответствовать базовым требованиям доступности (WCAG) в пределах, допустимых объёмом проекта [16]. Поддержка современных браузеров и их версий должна быть явно зафиксирована в спецификации на этапе разработки и тестирования.

1.6. Выбор и обоснование технологического стека

Выбор технологического стека обусловлен функциональными и нефункциональными требованиями проекта: необходимостью быстрой разработки минимально жизнеспособного продукта (MVP), надёжного хранения структурированных данных, а также возможностью масштабируемого развёртывания с поддержкой работы в реальном времени и фоновой обработки задач.

Основной фреймворк: Django [7, 8]. Данное решение обосновано концепцией «batteries included» (всё необходимое включено в комплект). Встроенные механизмы аутентификации, управления правами доступа, защиты от распространённых веб-уязвимостей и готовая административная панель существенно сокращают объём типовой работы. Это позволяет разработчику сосредоточиться на предметной области – логике проведения викторин и игровых механиках. Фреймворк Django REST Framework обеспечивает удобную и расширяемую реализацию API для одностраничных приложений (SPA) или мобильных клиентов. Встроенная объектно-реляционная модель (ORM) и система миграций упрощают управление схемой данных в процессе развития проекта.

PostgreSQL [13] выбран в качестве основной СУБД исходя из требований к надёжности, целостности транзакций и аналитическим возможностям. Поддержка сложных запросов, различных типов индексов, расширений и типизированных полей (включая тип JSONB) делает PostgreSQL подходящим для хранения как классических реляционных сущностей (вопросы, попытки, результаты), так и гибких аналитических данных. Механизмы транзакций и контроля параллельного доступа важны для корректного подсчёта очков и учёта серий ответов в условиях конкурентных обновлений. Кроме того, PostgreSQL предоставляет инструменты для репликации и резервного копирования, необходимые для обеспечения доступности и сохранности данных.

Контейнеризация: Docker [9]. Docker включён в состав стека в качестве инструмента развёртывания и стандартизации окружения. Контейнеризация обеспечивает воспроизводимость среды разработки и продукционной среды, изоляцию зависимостей и предсказуемость поведения приложения при переносе между различными машинами. Для локальной разработки и непрерывной интеграции (CI) целесообразно использовать Docker Compose, позволяя описать в одном файле все необходимые сервисы: веб-приложение, базу данных, кэш и

брокер сообщений, фоновый обработчик задач, обратный прокси-сервер и сервисы мониторинга. Для продуктивной среды можно оркестраторы (Kubernetes или Docker Swarm) либо облачные платформенные сервисы, обеспечивающие автоматическое масштабирование, управление секретами и перезапуск контейнеров при сбоях. Использование многоступенчатой сборки (multi-stage) и минимальных базовых образов позволяет уменьшить размер образов и ускорить развёртывание.

Для обеспечения производительности возможно применить кэширование на нескольких уровнях: HTTP-кэширование, кэш в Redis для частых запросов и использование PgBouncer для пула соединений с базой данных. Для аналитики и агрегатов логично хранить детальные данные в PostgreSQL, а для быстрых выборок использовать материализованные представления или отдельные агрегированные таблицы, обновляемые через фоновые задачи.

Резервное копирование базы данных рекомендуется организовать с помощью `pg_dump` или `pg_basebackup` с хранением копий в удалённом объектном хранилище и регулярной проверкой восстановления. Логирование и мониторинг реализуются через связку Sentry для отслеживания ошибок приложения и Prometheus с Grafana для сбора метрик производительности. Централизованное хранение логов может быть организовано с использованием стека ELK или специализированного облачного сервиса.

Альтернативные решения (Node.js с Socket.io, Firebase, Ruby on Rails) обладают собственными преимуществами, однако в контексте образовательного проекта Django даёт выигрыш в скорости разработки, наличии готовых инструментов безопасности и встроенного административного интерфейса. Docker делает выбранный стек переносимым и готовым к автоматизации развёртывания.

Глава 2. Практическая часть

2.1. Проектирование базы данных

На диаграмме (рис. 2.1) представлены 10 сущностей, разделённых на четыре модуля системы – accounts, quiz, session, leaderboard.

Центральной сущностью всей системы является таблица `accounts_user`. Она расширяет стандартную модель пользователя Django и хранит не только аутентификационные данные (логин, адрес электронной почты, пароль), но и игровой профиль пользователя: накопленные очки, текущий уровень, количество дней подряд активности (серия) и статистику ответов. Такой подход позволяет обойтись без отдельной таблицы профиля.

Таблица `accounts_achievement` содержит каталог достижений. Каждое достижение описывается категорией, иконкой, наградой в очках и пороговым значением, необходимым для разблокировки. Связь между пользователем и достижениями реализована через промежуточную таблицу `accounts_userachievement`, которая фиксирует момент получения каждой награды. Отношение «многие ко многим» реализуется именно через эту таблицу.

Таблица `quiz_category` служит для группировки викторин по тематике. Каждая викторина (`quiz_quiz`) принадлежит ровно одной категории и одному создателю; связи «многие к одному» установлены с таблицами `quiz_category` и `accounts_user` соответственно.

Викторина состоит из вопросов (`quiz_question`). Каждый вопрос содержит несколько вариантов ответа (`quiz_answer`), причём ровно один из них помечен как правильный. Таким образом, реализована классическая двухуровневая иерархия «один ко многим»: викторина → вопросы → ответы.

Когда пользователь начинает викторину, создаётся запись в `quiz_quizsession`, которая связывает пользователя и конкретную викторину. В этой записи накапливаются итоговые показатели: общий счёт, количество правильных ответов, максимальная серия (стрик) подряд верных ответов и затраченное время.

Каждый отдельный ответ пользователя фиксируется в `quiz_useranswer`. Данная таблица является наиболее связанной в схеме: она одновременно ссылается на сессию, вопрос и выбранный вариант ответа, а также хранит флаг правильности и количество заработанных очков. Такая структура позволяет восстановить

Листинг 1 – Структура веб-приложения

```
quiz_app/
├── config/                # Конфигурация проекта
│   ├── settings.py       # Настройки
│   ├── urls.py           # Корневой маршрутизатор
│   └── wsgi.py           # Точка входа WSGI-сервера
│
├── accounts/             # Модуль пользователей
│   ├── models.py         # Модели модуля пользователей
│   ├── views.py          # Регистрация, вход, профиль
│   ├── forms.py          # Формы
│   ├── urls.py           # Маршруты /accounts/
│   └── admin.py          # Управления пользователями
│
├── quiz/                 # Модуль викторин
│   ├── models.py         # Модели модуля викторин
│   ├── views.py          # Отображение элементов
│   ├── urls.py           # Маршруты /quizzes/, /play/
│   └── admin.py          # Управление викторинами
│
├── leaderboard/          # Модуль рейтинга
│   ├── models.py         # LeaderboardEntry
│   ├── views.py          # Таблица лидеров
│   └── urls.py           # Маршрут /leaderboard/
│
├── templates/            # HTML-шаблоны
│   ├── base.html         # Базовый макет
│   ├── accounts/
│   │   ├── login.html
│   │   ├── register.html
│   │   ├── profile.html
│   │   ├── edit_profile.html
│   │   └── public_profile.html
```



```

|   └─ quiz/
|   │   └─ home.html    # Главная
|   │   └─ list.html    # Список с фильтрами
|   │   └─ detail.html  # Карточка викторины
|   │   └─ play.html    # Игровой экран
|   │   └─ result.html  # Результаты
|   └─ leaderboard/
|       └─ leaderboard.html
|
└─ static/
    └─ css/main.css      # Все стили
    └─ js/main.js        # Вспомогательные скрипты
|
└─ seed_data.py          # Демо-версия БД
└─ requirements.txt       # Зависимости Python
└─ Dockerfile            # Образ контейнера
└─ docker-compose.yml    # Оркестрация web + db
└─ .env.example          # Пример переменных окружения

```

Модуль `config` содержит глобальные настройки проекта. В `settings.py` вынесены параметры подключения к PostgreSQL через переменные окружения, пути к статическим и медиафайлам, а также константы геймификации – очки за правильный ответ, бонус за скорость ответа и бонус за серию верных ответов.

Модуль `accounts` отвечает за весь жизненный цикл пользователя: регистрацию, аутентификацию, хранение игрового профиля. Пользовательская модель `User` наследует `AbstractUser` и расширена полями геймификации. Логика начисления уровней инкапсулирована в методах модели (`update_level`, `level_progress`), что избавляет представления от избыточных вычислений.

Модуль `quiz` представляет собой ядро системы. Представления реализуют полный цикл прохождения викторины:

- создание сессии;
- страничный показ вопросов с таймером;
- AJAX-обработка ответа (функция `submit_answer_view`);

- финализация сессии с обновлением статистики;
- экран с результатами.

Проверка достижений (`_check_achievements`) и обновление статистики у пользователя (`_update_user_stats`) вызываются автоматически при завершении сессии.

Модуль `leaderboard` намеренно оставлен лёгким: он не дублирует данные, а агрегирует их непосредственно из таблицы `accounts_user`, выстраивая рейтинг по полю `total_points`.

Шаблоны построены на механизме наследования от `base.html`, который обеспечивает единый макет, навигацию и вывод системных сообщений. Игровой экран (`play.html`) содержит встроенный JavaScript, реализующий:

- таймер обратного отсчёта;
- отправку ответа через Fetch API без перезагрузки страницы;
- анимацию начисленных очков.

Скрипт `seed_data.py` при первом запуске выполняет следующие действия:

- создание суперпользователя;
- создание демонстрационного аккаунта;
- заполнение набора достижений;
- создание категорий;
- создание трёх готовых викторин с вопросами.

Благодаря этому система сразу готова к демонстрации без необходимости ручного ввода данных через административную панель.

2.3. Описание реализованного функционала

Аутентификация построена на основе модуля `django.contrib.auth` с расширением через пользовательскую модель `User`, которая унаследована от `AbstractUser`. В системе реализованы следующие возможности:

- Для регистрации используется форма `RegisterForm` построенная на основе `UserCreationForm` с проверкой уникальности логина, корректности адреса электронной почты и надёжности пароля. При успешной регистрации пользователь автоматически авторизуется в системе;
- Для входа и выхода пользователя применяется форма `LoginForm` на ос-

нове `AuthenticationForm`. Выход из системы завершает сессию и перенаправляет пользователя на страницу входа;

- Все страницы, требующие авторизации, специально были защищены декоратором `@login_required` с автоматическим перенаправлением неавторизованных пользователей на форму входа;
- Пользователь имеет возможность изменить имя, адрес электронной почты, биографию и загрузить аватар;
- Страница любого пользователя является публичной, доступна по адресу `/accounts/profile/<username>/` и отображает статистику и достижения без возможности редактирования.

В модуле викторин реализован полный цикл прохождения викторины – от выбора до просмотра результатов.

Каталог викторин поддерживает фильтрацию одновременно по категории и уровню сложности (лёгкий, средний, сложный). Карточка каждой викторины отображает категорию, сложность, количество вопросов, лимит времени и число прохождений.

Запуск викторины создаёт объект `QuizSession`, который фиксирует момент начала и привязывает прохождение к конкретному пользователю. Счётчик прохождений викторины (`play_count`) увеличивается при каждом запуске.

Игровой экран реализован без перезагрузки страницы:

- Таймер обратного отсчёта отрисован средствами JavaScript и визуализирован в виде прогресс-бара, который меняет цвет с зелёного на красный при остатке времени менее 5 секунд.
- При истечении времени ответ засчитывается автоматически как пропущенный.
- Выбранный вариант отправляется на сервер через Fetch API (POST-запрос на `/play/<id>/submit/`). В ответ возвращается JSON-объект с результатом, правильным ответом и пояснением, которые отображаются немедленно без перехода на новую страницу.

Экран результатов показывает итоговую оценку по пятибалльной шкале; набранные очки, точность ответов, количество правильных ответов и максимальную серию (стрик); полный разбор всех вопросов с указанием выбранного и правильного ответов, а также пояснений (там, где они заданы).

Начисление очков происходит по следующей схеме (таблица 1):

Таблица 1 – Соответствие события и награды в виде очков

Событие	Очки
Правильный ответ	+10
Быстрый ответ (менее 5 секунд)	+3
Серия (каждые 3 правильных ответа подряд)	+5
Получение достижения	от +50 до +250

Система включает 7 уровней с названиями от «Новичок» до «Легенда». Переход на следующий уровень происходит автоматически при достижении порогового значения очков. На странице профиля отображается прогресс-бар, показывающий продвижение к следующему уровню.

Стрики отслеживаются в двух измерениях:

- серия правильных ответов внутри одной викторины (влияет на бонусные очки);
- серия дней подряд с активностью (хранится в профиле пользователя).

Реализовано 8 достижений четырёх категорий: за количество пройденных викторин, за дневные серии, за накопленные очки и за точность ответов. Проверка условий разблокировки выполняется автоматически после каждого завершённого прохождения. При получении нового достижения на экране результатов появляется анимированное всплывающее окно с названием, описанием и размером награды.

На странице профиля агрегируется полная статистика пользователя:

- суммарные очки;
- количество пройденных викторин;
- процент правильных ответов;
- текущая дневная серия.

Также отображаются все полученные достижения и история последних пяти прохождений с указанием даты, набранных очков и точности ответов.

Глобальный рейтинг формируется на основе суммарных очков всех пользователей. Первые три места отмечены медалями. Для авторизованного пользователя подсвечивается его строка и отображается текущая позиция в рейтинге. По каждому участнику видны уровень, количество пройденных викторин и точность ответов.

Стандартная панель Django Admin расширена для удобного управления контентом. Администратор имеет возможность управлять пользователями, категориями, достижениями и просматривать историю всех сессий.

Проект контейнеризован с помощью Docker Compose и включает два сервиса:

- db – база данных на основе PostgreSQL 15;
- web – приложение на Python 3.12.10.

Механизм healthcheck гарантирует, что Django не запускается до полной готовности базы данных. Миграции применяются автоматически при запуске контейнера. Скрипт `seed_data.py` заполняет базу данных демонстрационными данными за один запуск.

2.4. Демонстрация пользовательского интерфейса

Главная страница (рис. 2.2) является точкой входа в приложение и состоит из следующих блоков:

- Приветственный баннер с заголовком, кратким описанием платформы и двумя кнопками призыва к действию. Для авторизованных пользователей кнопки ведут на каталог викторин и таблицу лидеров; для неавторизованных – на страницы регистрации и входа;
- Три статистических показателя платформы в реальном времени: общее количество викторин, число зарегистрированных игроков и количество завершённых прохождений;
- Сетка карточек всех категорий с иконкой и счётчиком викторин внутри каждой. Клик по категории открывает каталог, отфильтрованный по выбранной тематике;
- Шесть карточек наиболее часто проходимых викторин с кнопкой перехода к полному каталогу;
- Пятёрка лидеров по набранным очкам с медалями, аватарами и ссылками на публичные профили.

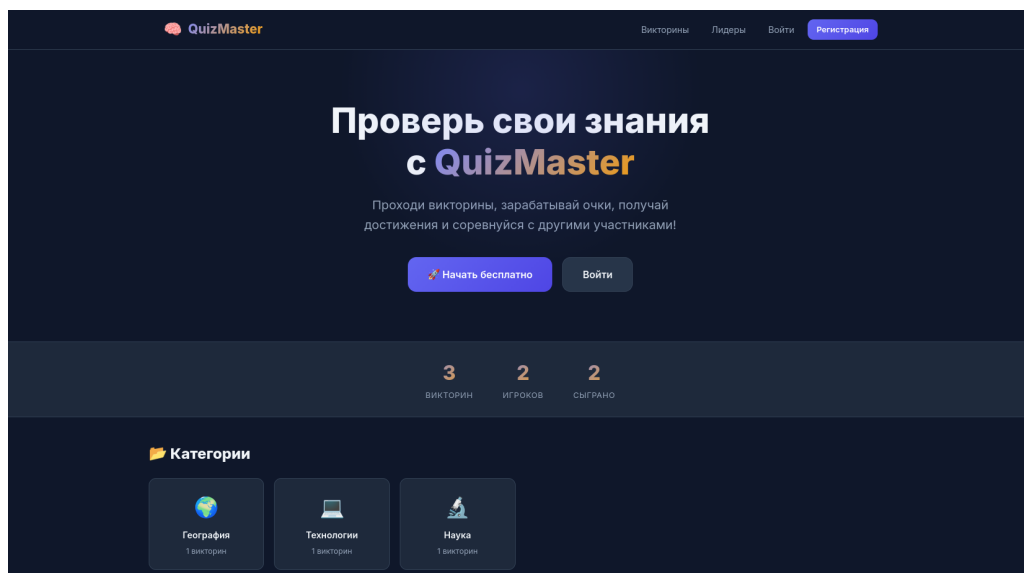


Рисунок 2.2 – Домашняя страница

Каталог (рис. 2.3) доступен только авторизованным пользователям и отображает все активные викторины в виде сетки карточек.

Панель фильтров позволяет комбинировать отбор одновременно по категории и уровню сложности. Каждая карточка содержит следующие сведения:

- категория;
- уровень сложности с цветовой кодировкой (зелёный – лёгкий, жёлтый – средний, красный – сложный);
- название и описание;
- количество вопросов;
- лимит времени;
- число прохождений.

При отсутствии результатов, соответствующих выбранным фильтрам, отображается заглушка с предложением изменить параметры отбора.

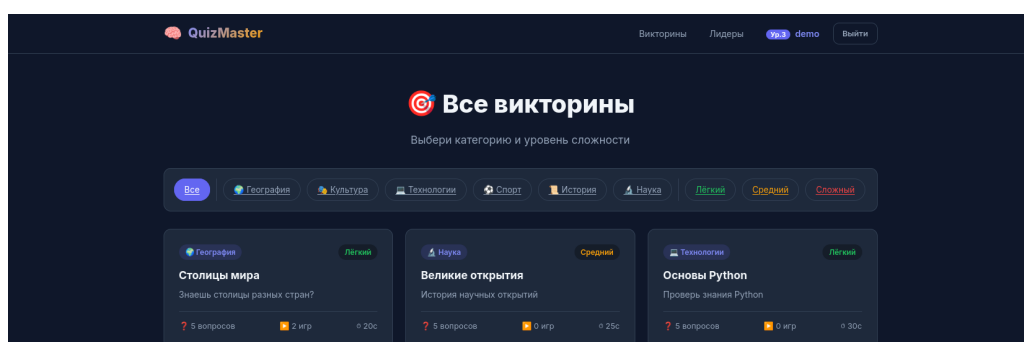


Рисунок 2.3 – Каталог викторин

Детальная страница (рис. 2.4), отображаемая перед началом прохождения. Содержит полное описание викторины, метаданные (категория, сложность, количество вопросов, лимит времени, автор, число прохождений) и блок запуска.

Для пользователей, уже проходивших данную викторину, в блоке запуска отображается их лучший результат. Ниже расположена таблица последних трёх прохождений с указанием даты, набранных очков, точности и максимальной серии (стрика).

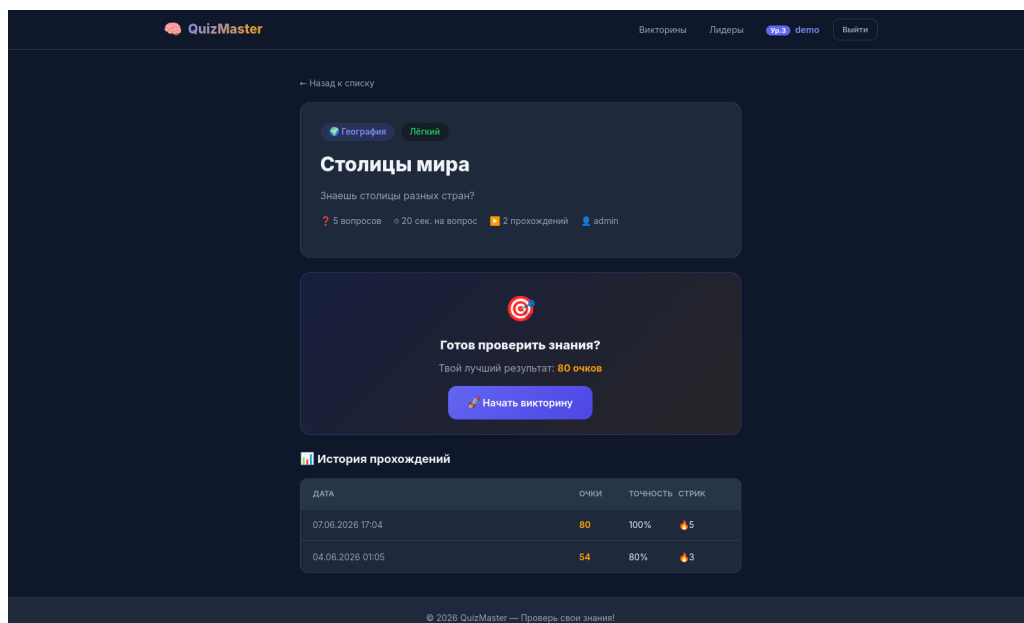


Рисунок 2.4 – Страница с подробным описанием викторины

Основной экран взаимодействия (прохождения викторины) (рис. 2.5). Интерфейс разделён на три функциональные зоны:

- Шапка содержит прогресс-бар прохождения с номером текущего вопроса, счётчик текущей серии правильных ответов и табло с накопленными очками. Все три элемента обновляются в реальном времени без перезагрузки страницы;
- Таймер – горизонтальная полоса, убывающая каждую секунду. Меняет цвет с зелёного на красный, если осталось менее 5 секунд. При достижении нуля ответ автоматически засчитывается как пропущенный;
- Отображение текста вопроса и четыре варианта ответа в виде кнопок. После выбора все кнопки блокируются; правильный ответ подсвечивается зелёным цветом, а неверно выбранный – красным. Если для вопроса задано пояснение, оно появляется в отдельном блоке. При правильном ответе над экраном всплывает анимация с количеством заработанных очков. Кнопка перехода к следующему вопросу появляется только после того, как дан ответ на текущий.

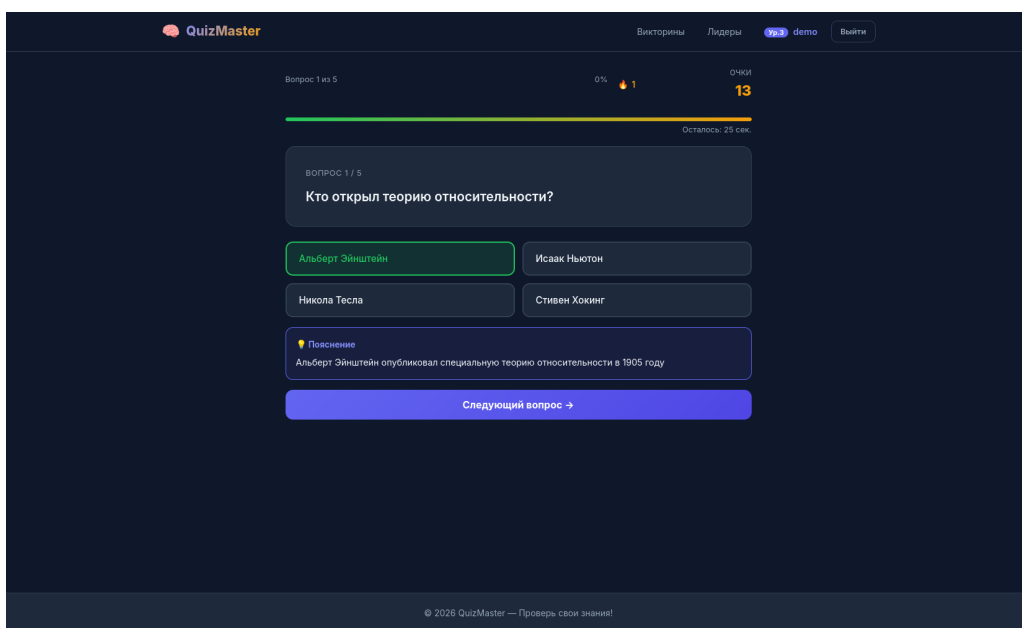


Рисунок 2.5 – Страница прохождения викторины

Страница с результатами прохождения викторины (рис. 2.6) отображается после завершения последнего вопроса и содержит следующие элементы:

- Итоговая карточка. Показывает буквенную оценку (S/A/B/C/D) с цветовой индикацией, заголовок с поздравлением в зависимости от достигнутой точности и плашку «Лучший результат» при установке нового рекорда. Под оценкой расположены четыре показателя: набранные очки, точность в процентах, соот-

ношение правильных ответов к общему числу, максимальная серия;

- Разбор ответов. Список всех вопросов с указанием выбранного и правильного вариантов (выделенных цветом), а также пояснением при его наличии;
- Блок новых достижений. Если по итогам прохождения разблокировано одно или несколько достижений, поверх страницы последовательно появляются всплывающие окна с иконкой, названием, описанием и размером награды;
- Кнопки действий. Позволяют пройти викторину заново, перейти к каталогу или открыть таблицу лидеров.

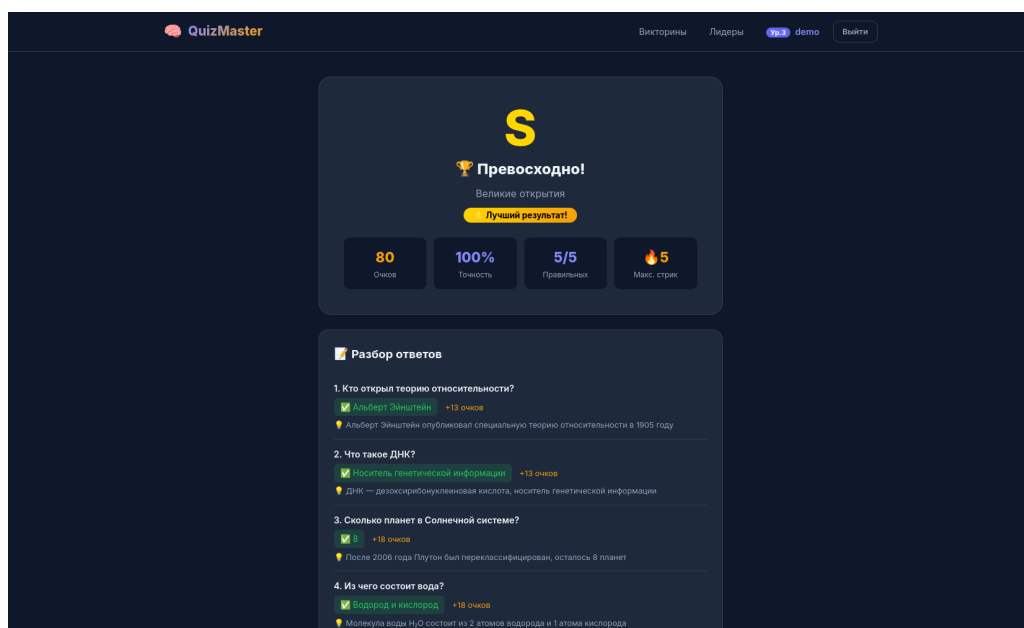


Рисунок 2.6 – Страница с результатами прохождения викторины

Страница входа содержит форму с полями для ввода логина и пароля. При ошибке аутентификации выводится соответствующее сообщение под формой. После успешного входа пользователь перенаправляется на главную страницу или на запрошенный URL при наличии параметра `next`.

Страница регистрации (рис. 2.7) содержит форму с полями имени пользователя, адреса электронной почты, пароля и его подтверждения. Ошибки валидации отображаются под каждым полем. При успешной регистрации пользователь автоматически авторизуется и перенаправляется на главную страницу.

QuizMaster

Викторины Лидеры Войти **Регистрация**

Создать аккаунт
Присоединяйся к тысячам игроков!

USERNAME
Имя пользователя

EMAIL
your@email.com

PASSWORD1
Пароль

PASSWORD2
Повтор пароля

Зарегистрироваться

Уже есть аккаунт?
Войти

© 2026 QuizMaster — Проверь свои знания!

Рисунок 2.7 – Форма регистрации

Личный профиль (рис. 2.8) доступен только владельцу учётной записи. Данная страница имеет следующую структуру:

- Верхняя часть отображает аватар, имя пользователя, текущий уровень с названием и прогресс-бар до следующего уровня;
- Блок статистики включает четыре карточки с показателями: суммарные очки, количество пройденных викторин, общая точность и дневная серия;
- Сетка полученных достижений с иконками и описаниями;
- История прохождений из таблицы пяти последних прохождений;
- Кнопка редактирования ведёт на форму изменения профиля.

QuizMaster

Викторины Лидеры **demo** **Войти**

demo
Уровень 3 Знаток
До следующего уровня 38%

414 Очков

3 Пройдено

93,3% Точность

2 Дней подряд

Достижения (3)

Снайпер
Достигли точности 80%
+100 очков

Первые очки
Набрали 100 очков
+50 очков

Первый шаг
Пройди первую викторину
+50 очков

Последние прохождения

Викторина	Очки	Точность	Стрик
Великие открытия 06.06.2026	80	100%	5
Столицы мира 07.06.2026	80	100%	5

Рисунок 2.8 – Личный профиль пользователя

Редактирование профиля (рис. 2.9) включает форму с полями имени, фамилии, адреса электронной почты, биографии и загрузки аватара. Поддерживает передачу изображений через `multipart/form-data`. После сохранения изменений пользователь перенаправляется обратно в профиль с уведомлением об успешном обновлении.

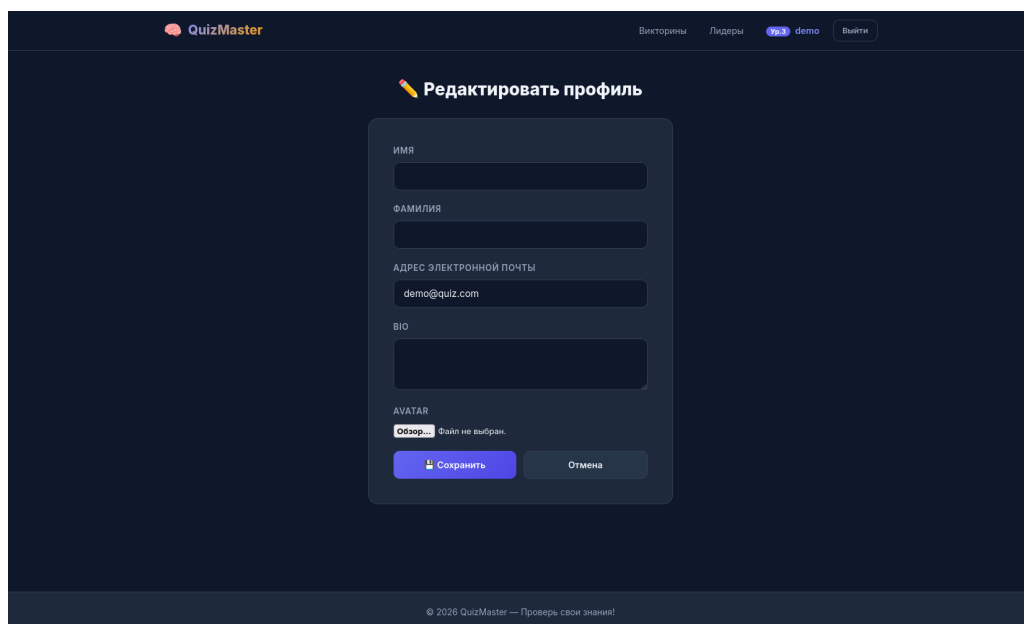


Рисунок 2.9 – Страница с редактированием профиля

Страница любого пользователя доступна без авторизации. Содержит те же блоки, что и личный профиль (аватар, уровень, статистику, достижения и историю прохождений), но без возможности редактирования.

Рейтинговая таблица (рис. 2.10), отображающая двадцать игроков с наибольшим количеством очков. Колонки таблицы:

- место;
- аватар и имя пользователя со ссылкой на профиль;
- уровень;
- суммарные очки;
- количество пройденных викторин;
- общая точность.

Первые три строки отмечены золотой, серебряной и бронзовой медалями соответственно. Строка текущего авторизованного пользователя выделена цветом; его позиция и количество очков дублируются над таблицей.

#	ИГРОК	ОЧКИ	ПРОЙДЕНО	ТОЧНОСТЬ
1	demo Знаток Ур.3	414	3 викторины	93.3%
2	admin Новичок Ур.1	0	0 викторины	0%

Рисунок 2.10 – Таблица лидеров

Стандартный интерфейс Django Admin (рис. 2.11), расширенный для удобного управления всеми моделями. Панель позволяет:

- создавать викторины вместе с вопросами и ответами на одной странице;
- управлять пользователями и достижениями;
- просматривать историю всех игровых сессий.

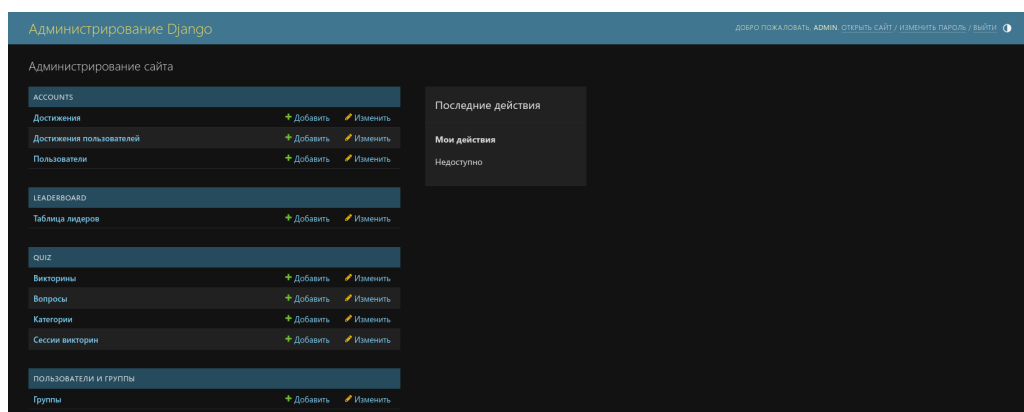


Рисунок 2.11 – Панель администратора

2.5. Что можно добавить в систему в будущем

В текущей реализации каждый запрос к таблице лидеров, главной странице и профилям пользователей выполняет повторные обращения к базе данных. Для устранения этого недостатка целесообразно внедрить многоуровневое кэширование.

В качестве бэкенда кэша Django предлагается использовать Redis, что позволит кэшировать результаты тяжёлых агрегирующих запросов – таблицу лидеров, статистику платформы, списки викторин по категориям – с автоматическим удалением кэша по истечении времени жизни (TTL) или при изменении

данных. Необходима также оптимизация ORM-запросов посредством расстановки методов `select_related` и `prefetch_related` там, где в настоящее время возникают N+1 запросов. Это касается, в частности, загрузки списка викторин с категориями и авторами, а также вывода достижений с их описаниями. Для страниц с низкой частотой изменений – главной страницы, каталога и публичных профилей – рекомендуется использовать пакет `django-cachalot` либо ручное кэширование через декоратор `@cache_page`.

Со стороны реализованной базы данных возможно добавить индексы на поля `total_points`, `created_at`, `is_active` и `category_id` для ускорения операций сортировки и фильтрации при увеличении объёма данных. Кроме того, каталог викторин и таблица лидеров должны использовать постраничную навигацию вместо загрузки всех записей одним запросом.

Текущая система достижений является статической: все достижения заданы заранее и проверяются по простым числовым порогам. Возможны следующие направления её углубления: введение ежедневных и еженедельных заданий с временными наградами; создание системы лиг, то есть разбивка игроков на дивизионы с периодическим сбросом рейтинга по окончании сезона; добавление значков за особые события, такие как первое место в таблице лидеров, серия из десяти правильных ответов подряд или прохождение всех викторин одной категории; а также введение виртуальной валюты для разблокировки тем оформления профиля.

Для повышения вовлечённости пользователей можно добавить ряд социальных возможностей. В частности, функция дуэли с другим пользователем позволит проводить синхронное прохождение одной викторины с отображением прогресса соперника в реальном времени через Websocket (с использованием `django-channels`). Также целесообразно добавить возможность комментирования и оценки викторин, систему подписок с лентой активности отслеживаемых игроков и возможность делиться результатом прохождения по ссылке.

В настоящее время создавать викторины может только администратор через панель `/admin/`. Внедрение пользовательского конструктора позволило бы участникам создавать викторины через удобный веб-интерфейс с добавлением вопросов и вариантов ответов. Требуется поддержка разных типов вопросов: с одним вариантом ответа, с несколькими вариантами, с вводом текста, на соответствие. Кроме того, конструктор должен обеспечивать возможность

добавления изображений к вопросам и вариантам ответов, настройку уровня доступа (публичная викторина, доступная по ссылке или приватная), а также просмотр статистики по созданным викторинам.

Также можно добавить аналитику. Для обычных пользователей – личный дашборд с графиками прогресса по времени: рост очков, динамика точности, активность по дням, а также тепловая карта активности в стиле вкладов GitHub и детальная статистика по категориям для определения тем, в которых пользователь силён, и тем, которые требуют изучения. Для администратора предусматривается аналитическая панель с метриками платформы: дневная и месячная активная аудитория (DAU/MAU), конверсия регистраций, самые популярные и самые сложные вопросы.

Можно добавить систему уведомлений с тремя каналами доставки. Email-уведомления отправляются при получении нового достижения, при вызове на дуэль и при обгоне в таблице лидеров. Push-уведомления в браузере, реализуемые через Web Push API, используются для напоминания о ежедневной активности и поддержания серии (стрика). Внутренняя система уведомлений отображается в виде значка-колокольчика в навигационной панели.

Текущий интерфейс является адаптивным, однако полноценное мобильное решение требует следующих доработок. Приложение должно быть оформлено как прогрессивное веб-приложение (Progressive Web App) с манифестом и сервис-воркером для работы в автономном режиме. Необходимо обеспечить сохранение прогресса прохождения при потере соединения с последующей синхронизацией после восстановления связи. Также следует реализовать использование нативных жестов и анимаций для мобильного игрового экрана.

Для обеспечения безопасности и возможности горизонтального масштабирования возможно добавит ограничение частоты запросов с использованием пакета `django-ratelimit` защитит формы авторизации от атак подбора паролей. Двухфакторная аутентификация должна быть реализована через TOTP (Time-based One-Time Password). Горизонтальное масштабирование включает вынос статических файлов на CDN, настройку Gunicorn с несколькими воркерами и установку балансировщика Nginx перед приложением. Наконец, для асинхронной обработки тяжёлых задач – пересчёта рейтинга, отправки электронных писем и генерации отчётов – предлагается использовать связку Celery и Redis.

Заключение

В ходе выполнения курсового проекта по дисциплине «Методы и средства проектирования информационных систем и технологий» было спроектировано и разработано веб-приложение QuizMaster – платформа для проведения викторин с элементами геймификации.

В теоретической части работы были рассмотрены принципы построения веб-приложений как класса информационных систем, изучены современные методы и средства их проектирования, проанализировано понятие геймификации и её применение в образовательных системах. Проведён анализ предметной области и сформулированы функциональные и нефункциональные требования к системе. На основании проведённого анализа был обоснован выбор технологического стека: фреймворк Django обеспечивает структурированную архитектуру по паттерну MTV и развитую экосистему готовых компонентов, а СУБД PostgreSQL – надёжное реляционное хранилище с поддержкой сложных запросов и индексирования.

В практической части были решены все задачи, поставленные во введении. Спроектирована реляционная база данных, включающая десять взаимосвязанных таблиц, охватывающих управление пользователями, структуру викторин, фиксацию игровых сессий и рейтинговую систему. Разработана трёхуровневая архитектура приложения с разделением на модули `accounts`, `quiz` и `leaderboard`. Реализована система аутентификации и авторизации с пользовательской моделью пользователя. Разработан модуль викторин с поддержкой категорий, уровней сложности и таймера ответа. Реализована многокомпонентная система геймификации, включающая начисление очков с бонусами за скорость и серии, семь уровней пользователя, восемь достижений и таблицу лидеров. Разработан пользовательский интерфейс, содержащий двенадцать страниц с тёмной темой оформления и адаптивной вёрсткой. Приложение контейнеризовано с использованием Docker Compose для воспроизводимого развёртывания.

Практическая значимость разработанной системы состоит в том, что оно представляет собой готовое к использованию веб-приложение, которое может быть применено в образовательных учреждениях для интерактивной проверки знаний, в корпоративной среде для обучения сотрудников, а также как самосто-

ятельная платформа для широкой аудитории.

В процессе выполнения работы были получены практические навыки проектирования реляционных баз данных с использованием ER-диаграмм, разработки веб-приложений на основе фреймворка Django, организации пользовательской аутентификации, реализации игровых механик в информационных системах, а также контейнеризации приложений средствами Docker.

Среди перспективных направлений дальнейшего развития системы следует выделить внедрение кэширования на основе Redis для оптимизации производительности при росте нагрузки, добавление конструктора викторин для пользователей, реализацию социальных функций и расширение аналитических инструментов.

Таким образом, все задачи курсового проекта были выполнены в полном объёме, а поставленная цель – разработка веб-приложения для проведения викторин с элементами геймификации – достигнута.

Список использованных источников

1. Кольцов, Д.А. Справочник Python. Кратко, быстро, под рукой / Д.А. Кольцов, Е. Дубовик. — Санкт-Петербург : Наука и Техника, 2021. — 288 с (дата обращения: 20.05.2026).
2. Лутц, М. Программирование на Python. Т. 1 / М. Лутц. — 4-е изд. — Санкт-Петербург : Символ-Плюс, 2011. — 992 с (дата обращения: 19.05.2026).
3. Федоров, Д.Ю. Программирование на языке высокого уровня Python : учебное пособие для прикладного бакалавриата / Д.Ю. Федоров. — Москва : Юрайт, 2017. — 126 с (дата обращения: 25.05.2026).
4. Чернышев, С.А. Основы программирования на Python : учебное пособие для вузов / С.А. Чернышев. — Москва : Юрайт, 2023. — 286 с (дата обращения: 25.05.2026).
5. Csikszentmihalyi, M. Flow: The Psychology of Optimal Experience / M. Csikszentmihalyi. — New York : Harper & Row, 1990 (дата обращения: 27.05.2026).
6. Deterding, S. From game design elements to gamefulness: defining "gamification" / S. Deterding, D. Dixon, R. Khaled, L. Nacke // Proceedings of the 15th International Academic MindTrek Conference. — 2011. — P. 9–15 (дата обращения: 26.05.2026).
7. Django Documentation. — URL: <https://docs.djangoproject.com/> (дата обращения: 22.05.2026).
8. Django REST Framework Documentation. — URL: <https://www.django-rest-framework.org/> (дата обращения: 22.05.2026).
9. Docker Official Documentation. — URL: <https://docs.docker.com/> (дата обращения: 23.05.2026).
10. Kahoot! Official Website. — URL: <https://kahoot.com/> (дата обращения: 27.05.2026).

11. Mentimeter Official Website. — URL: <https://www.mentimeter.com/> (дата обращения: 27.05.2026).
12. Moodle Official Website. — URL: <https://moodle.org/> (дата обращения: 27.05.2026).
13. PostgreSQL Official Documentation. — URL: <https://www.postgresql.org/docs/> (дата обращения: 23.05.2026).
14. Quizizz Official Website. — URL: <https://quizizz.com/> (дата обращения: 27.05.2026).
15. Ryan, R.M. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being / R.M. Ryan, E.L. Deci // American Psychologist. — 2000. — Vol. 55, No. 1. — P. 68–78 (дата обращения: 26.05.2026).
16. WCAG (Web Content Accessibility Guidelines). — URL: <https://www.w3.org/WAI/standards-guidelines/wcag/> (дата обращения: 29.05.2026).

Приложения

Листинг 1.1 – Реализация моделей модуля accounts в языке Python

```
1 from django.contrib.auth.models import AbstractUser
2 from django.db import models
3
4
5 class User(AbstractUser):
6     avatar = models.ImageField(upload_to="avatars/", null=True,
7                               blank=True)
8     bio = models.TextField(max_length=500, blank=True)
9     total_points = models.IntegerField(default=0)
10    level = models.IntegerField(default=1)
11    streak_days = models.IntegerField(default=0)
12    last_activity = models.DateField(null=True, blank=True)
13    quizzes_completed = models.IntegerField(default=0)
14    correct_answers = models.IntegerField(default=0)
15    total_answers = models.IntegerField(default=0)
16    achievements_count = models.IntegerField(default=0)
17
18    class Meta:
19        verbose_name = "Пользователь"
20        verbose_name_plural = "Пользователи"
21
22    def __str__(self):
23        return self.username
24
25    @property
26    def accuracy(self):
27        if self.total_answers == 0:
28            return 0
29        return round((self.correct_answers / self.total_answers)
30                      * 100, 1)
31
32    @property
33    def level_name(self):
34        levels = {
35            1: "Новичок",
36            2: "Ученик",
37            3: "Знаток",
```

```

36         4: "Эксперт",
37         5: "Мастер",
38         6: "Гуру",
39         7: "Легенда",
40     }
41     return levels.get(min(self.level, 7), "Легенда")
42
43 @property
44 def points_to_next_level(self):
45     thresholds = [0, 100, 300, 600, 1000, 1500, 2200, 3000]
46     if self.level >= 7:
47         return 0
48     return thresholds[self.level] - self.total_points
49
50 @property
51 def level_progress(self):
52     thresholds = [0, 100, 300, 600, 1000, 1500, 2200, 3000]
53     if self.level >= 7:
54         return 100
55     prev = thresholds[self.level - 1]
56     curr = thresholds[self.level]
57     return int(((self.total_points - prev) / (curr - prev)) *
58                100)
59
60 def update_level(self):
61     thresholds = [0, 100, 300, 600, 1000, 1500, 2200, 3000]
62     for i, threshold in enumerate(thresholds):
63         if self.total_points >= threshold:
64             self.level = i + 1
65             self.save(update_fields=["level"])
66
67 class Achievement(models.Model):
68     CATEGORY_CHOICES = [
69         ("quiz", "Викторины"),
70         ("streak", "Серии"),
71         ("score", "Очки"),
72         ("accuracy", "Точность"),
73     ]
74     name = models.CharField(max_length=100)
75     description = models.TextField()

```

```

76 icon = models.CharField(max_length=10, default="□")
77 category = models.CharField(max_length=20, choices=
    CATEGORY_CHOICES, default="quiz")
78 points_reward = models.IntegerField(default=50)
79 requirement_value = models.IntegerField(default=1)
80
81 class Meta:
82     verbose_name = "Достижение"
83     verbose_name_plural = "Достижения"
84
85 def __str__(self):
86     return self.name
87
88
89 class UserAchievement(models.Model):
90     user = models.ForeignKey(
91         User, on_delete=models.CASCADE, related_name="
            user_achievements"
92     )
93     achievement = models.ForeignKey(Achievement, on_delete=
        models.CASCADE)
94     earned_at = models.DateTimeField(auto_now_add=True)
95
96 class Meta:
97     unique_together = ("user", "achievement")
98     verbose_name = "Достижение пользователя"
99     verbose_name_plural = "Достижения пользователей"

```

Листинг 1.2 – Реализация моделей модуля quiz в языке Python

```
1 from django.db import models
2 from django.conf import settings
3
4
5 class Category(models.Model):
6     name = models.CharField(max_length=100)
7     description = models.TextField(blank=True)
8     icon = models.CharField(max_length=10, default="□")
9     color = models.CharField(max_length=7, default="#6366f1")
10
11     class Meta:
12         verbose_name = "Категория"
13         verbose_name_plural = "Категории"
14
15     def __str__(self):
16         return self.name
17
18
19 class Quiz(models.Model):
20     DIFFICULTY_CHOICES = [
21         ("easy", "Лёгкий"),
22         ("medium", "Средний"),
23         ("hard", "Сложный"),
24     ]
25     title = models.CharField(max_length=200)
26     description = models.TextField()
27     category = models.ForeignKey(
28         Category, on_delete=models.CASCADE, related_name="quizzes"
29     )
30     difficulty = models.CharField(
31         max_length=10, choices=DIFFICULTY_CHOICES, default="medium"
32     )
33     time_limit = models.IntegerField(default=30, help_text="Секунд на вопрос")
34     created_by = models.ForeignKey(settings.AUTH_USER_MODEL,
35                                   on_delete=models.CASCADE)
36     created_at = models.DateTimeField(auto_now_add=True)
```

```

36 is_active = models.BooleanField(default=True)
37 play_count = models.IntegerField(default=0)
38
39 class Meta:
40     verbose_name = "Викторина"
41     verbose_name_plural = "Викторины"
42     ordering = ["-created_at"]
43
44 def __str__(self):
45     return self.title
46
47 @property
48 def question_count(self):
49     return self.questions.count()
50
51 @property
52 def difficulty_color(self):
53     colors = {"easy": "#22c55e", "medium": "#f59e0b", "hard":
54               "#ef4444"}
55     return colors.get(self.difficulty, "#6366f1")
56
57 @property
58 def difficulty_label(self):
59     labels = {"easy": "Лёгкий", "medium": "Средний", "hard":
60               "Сложный"}
61     return labels.get(self.difficulty, "Средний")
62
63 class Question(models.Model):
64     quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE,
65                             related_name="questions")
66     text = models.TextField()
67     explanation = models.TextField(
68         blank=True, help_text="Объяснение правильного ответа"
69     )
70     order = models.IntegerField(default=0)
71     points = models.IntegerField(default=10)
72
73     class Meta:
74         verbose_name = "Вопрос"
75         verbose_name_plural = "Вопросы"

```

```

74     ordering = ["order"]
75
76     def __str__(self):
77         return self.text[:80]
78
79
80 class Answer(models.Model):
81     question = models.ForeignKey(
82         Question, on_delete=models.CASCADE, related_name="answers
83         "
84     )
85     text = models.CharField(max_length=500)
86     is_correct = models.BooleanField(default=False)
87
88     class Meta:
89         verbose_name = "Ответ"
90         verbose_name_plural = "Ответы"
91
92     def __str__(self):
93         return f"{self.text} {' ' if self.is_correct else ''}"
94
95 class QuizSession(models.Model):
96     user = models.ForeignKey(
97         settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
98         related_name="sessions"
99     )
100     quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE,
101                             related_name="sessions")
102     started_at = models.DateTimeField(auto_now_add=True)
103     finished_at = models.DateTimeField(null=True, blank=True)
104     score = models.IntegerField(default=0)
105     correct_count = models.IntegerField(default=0)
106     total_questions = models.IntegerField(default=0)
107     current_question = models.IntegerField(default=0)
108     streak = models.IntegerField(default=0)
109     max_streak = models.IntegerField(default=0)
110     is_completed = models.BooleanField(default=False)
111     time_spent = models.IntegerField(default=0, help_text="
    Секунд")

```



```

111 class Meta:
112     verbose_name = "Сессия викторины"
113     verbose_name_plural = "Сессии викторин"
114     ordering = ["-started_at"]
115
116     def __str__(self):
117         return f"{self.user.username} -{self.quiz.title}"
118
119     @property
120     def accuracy(self):
121         if self.total_questions == 0:
122             return 0
123         return round((self.correct_count / self.total_questions)
124                       * 100)
125
126     @property
127     def percentage(self):
128         return self.accuracy
129
130 class UserAnswer(models.Model):
131     session = models.ForeignKey(
132         QuizSession, on_delete=models.CASCADE, related_name="
133         user_answers"
134     )
135     question = models.ForeignKey(Question, on_delete=models.
136         CASCADE)
137     answer = models.ForeignKey(Answer, on_delete=models.CASCADE,
138         null=True, blank=True)
139     is_correct = models.BooleanField(default=False)
140     time_taken = models.IntegerField(default=0, help_text="
141         Секунд")
142     points_earned = models.IntegerField(default=0)
143
144     class Meta:
145         verbose_name = "Ответ пользователя"
146         verbose_name_plural = "Ответы пользователей"

```

Листинг 1.3 – Реализация моделей модуля leaderboard в языке Python

```
1 from django.db import models
2 from django.conf import settings
3
4
5 class LeaderboardEntry(models.Model):
6     PERIOD_CHOICES = [
7         ("all", "За всёвремя"),
8         ("month", "За месяц"),
9         ("week", "За неделю"),
10    ]
11    user = models.ForeignKey(
12        settings.AUTH_USER_MODEL,
13        on_delete=models.CASCADE,
14        related_name="leaderboard_entries",
15    )
16    period = models.CharField(max_length=10, choices=
17        PERIOD_CHOICES, default="all")
18    points = models.IntegerField(default=0)
19    rank = models.IntegerField(default=0)
20    quizzes_completed = models.IntegerField(default=0)
21    updated_at = models.DateTimeField(auto_now=True)
22
23 class Meta:
24     unique_together = ("user", "period")
25     ordering = ["rank"]
26     verbose_name = "Запись таблицылидеров"
27     verbose_name_plural = "Таблица лидеров"
28
29 def __str__(self):
30     return f"{self.rank}. {self.user.username} –{self.points}
31         очков"
```